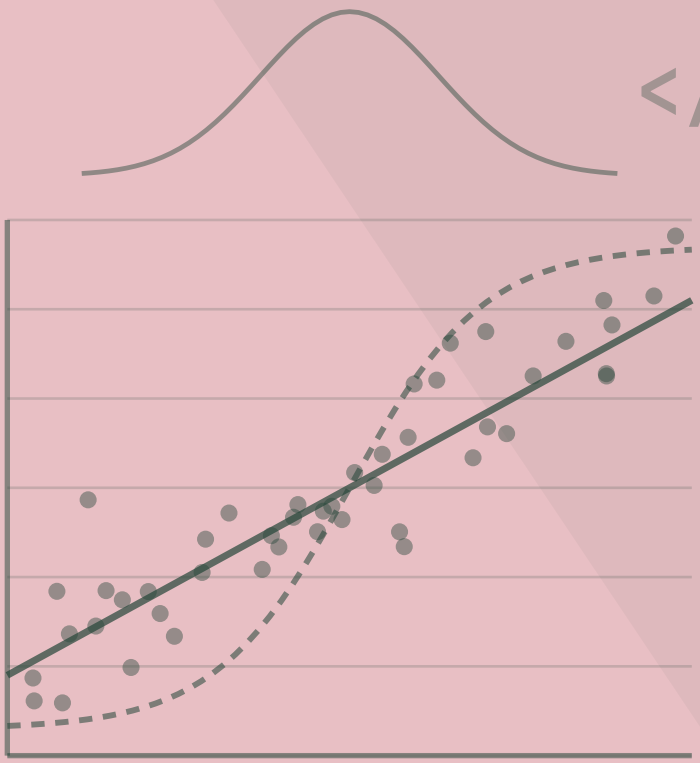
VENVLEARNING

STUDENT TEXTBOOK



```
proc means;    df.describe()    summary(x)
```

Data Science & Psychometrics

YEAR 1

Statistics, coding, and measurement in 30 minutes a day

2026–2027 Professional Learning · 2 units

A VenvLearning professional learning curriculum

SAS · PYTHON · R

VENVLEARNING

Professional Learning
Data Science & Psychometrics

STUDENT TEXTBOOK

2026–2027 Professional Learning

Unit 1 Your Data Toolkit
Unit 2 Data Wrangling

This book belongs to _____

A VenvLearning professional learning curriculum

How to use this book

This is a year of professional learning built to fit inside a workday: about 30 minutes, Monday through Friday. Monday through Thursday is a lesson. Friday is a short quiz plus time to use the week's skill on real work.

- A 30-minute day: Read 10 · Watch or Code 15 · Quick Check 5. If a day gets away from you, the Quick Check alone keeps you moving.
- Code It boxes show the same task in SAS, Python, and R. Run at least one language every day, and rotate so you build all three.
- Watch and Explore boxes suggest free videos and courses, plus optional DataCamp courses if you subscribe.
- On the Job boxes connect each skill to assessment, research, and reporting work.
- Every exercise uses the VenvLearning data pack (exam.csv, survey.csv, and more), so your results match the answer key.
- Your practice pages, Friday quizzes, unit exams, and answer key are in your Workbook.

Contents

Your Data Toolkit 4

Lesson 1.1: Your Toolkit and the Analysis Workflow 6

Lesson 1.2: Reading Data into SAS, Python, and R 9

Lesson 1.3: Variables, Data Types, and Levels of Measurement 11

Lesson 1.4: Reproducible Work Habits 13

Unit 1 Study Guide 15

Data Wrangling 16

Lesson 2.1: Selecting Columns and Filtering Rows 18

Lesson 2.2: Creating Variables: Scoring a Test 20

Lesson 2.3: Sorting, Counting, and Grouped Summaries 22

Lesson 2.4: Missing Data: Finding and Handling It 24

Lesson 2.5: Joining Tables 26

Lesson 2.6: Reshaping Data: Wide and Long 28

Lesson 2.7: Cleaning Messy Data 30

Lesson 2.8: Duplicates and Validation Checks 32
Lesson 2.9: SQL for Analysts I: SELECT, WHERE, GROUP BY 34
Lesson 2.10: SQL for Analysts II: Joins and Subqueries 36
Lesson 2.11: Don't Repeat Yourself: Functions and Macros 37
Lesson 2.12: Putting It Together: A Scoring Pipeline 39
Lesson 2.13: Working with Dates 41
Lesson 2.14: Working with Text 42
Lesson 2.15: Codebooks and Data Documentation 43
Lesson 2.16: Wrangling Project: Enrollment Meets Scores 45
Unit 2 Study Guide 47

Glossary 48

UNIT 1

Your Data Toolkit

The analysis workflow; reading data in SAS, Python, and R; types and levels of measurement; reproducible habits.

A week to set up your tools and your habits. You'll open SAS Studio, RStudio, and Jupyter, read the same file in all three, learn how data types and levels of measurement shape every analysis, and set up a project the way experienced analysts do.

In this unit you will…

- Describe the analysis workflow.
- Import data in SAS, Python, and R and check it.
- Identify data types and levels of measurement.
- Organize reproducible projects and read a SAS log.

ON THE JOB

Most of the time you spend on an analysis goes into getting data in and checking it. These habits make every later skill faster and more trustworthy.

UNIT VOCABULARY

workflow	Steps from question to communication.
data frame	A table of rows and columns.
ordinal	Ordered categories.
reproducible	Can be rerun with the same result.

Lessons in this unit

Lesson	Title
1.1	Your Toolkit and the Analysis Workflow
1.2	Reading Data into SAS, Python, and R
1.3	Variables, Data Types, and Levels of Measurement

Lesson	Title
1.4	Reproducible Work Habits

UNIT 1 · LESSON 1.1 · Foundations · 30 min

Lesson 1.1: Your Toolkit and the Analysis Workflow

I can describe the steps of a data analysis and set up SAS, Python, and R so I can practice every day.

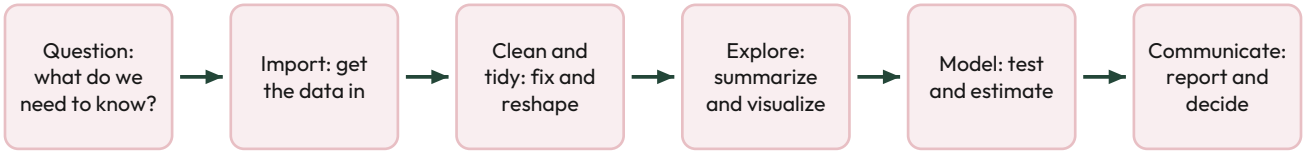
WORDS TO KNOW

workflow	The repeatable sequence of steps in an analysis, from question to communication.
reproducible	Able to be rerun by you or someone else and get the same result.
IDE	Integrated development environment: the program where you write and run code (SAS Studio, RStudio, Jupyter, VS Code).

WHY IT MATTERS AT WORK

Think of the last report or analysis you were handed at work. Where did the numbers come from, and could you have recreated them yourself?

Every analysis, from a quick table for a meeting to a full technical report on a test, follows roughly the same path. Naming the steps helps you see where you are confident and where you want to grow.



This year you'll practice each step in three languages. SAS is your workplace's statistics engine. Python is what you already use a lot. R is the language most psychometricians use for reliability, factor analysis, and item response theory. Seeing one task three ways makes each language easier, because you learn the idea first and the syntax second.

Tool	Free way to use it	What it's best at for you
SAS	SAS OnDemand for Academics (SAS Studio in a browser), or SAS at work	your team's production statistics; PROC IRT, PROC FACTOR, PROC CORR
Python	your current setup, or Anaconda / VS Code with Jupyter	data wrangling, automation, machine learning
R	R plus RStudio Desktop (Posit), or Posit Cloud in a browser	psychometrics packages: psych, mirt, lavaan

Your resource plan

Resource	Cost	Use it for
SAS OnDemand for Academics (SAS Studio)	free	running SAS anywhere; includes free e-learning: SAS Programming 1 and Statistics 1
R for Data Science (2e), r4ds.hadley.nz	free online	R and the tidyverse
Python for Data Analysis (3E), wesmckinney.com/book	free online	pandas
StatQuest with Josh Starmer (YouTube)	free	short, clear statistics videos
Introduction to Educational and Psychological Measurement Using R (Albano)	free online, CC BY	psychometrics in R (Units 6–8)
NCME ITEMS digital modules	check access on ncme.org	short modules on reliability, IRT, item analysis, standard setting
DataCamp Premium (optional)	about $25–27/month billed yearly, or about $42 month to month (check current price)	interactive R, Python, and SQL practice; Statistician (R) track; Factor Analysis in R; Structural Equation Modeling with lavaan in R

Is DataCamp worth renewing? It fits this course well for R, Python, and SQL practice, and the interactive format suits a 30-minute day. It has no SAS courses and only a few psychometrics courses, so this book carries the SAS and measurement content. A good approach: on Fridays, after your quiz, spend 15 minutes on the DataCamp chapter listed in that week's Watch and Explore box. If you'd use it at least twice a week, the annual plan is the better value; if not, the free resources above cover everything in this book.

CODE IT · HELLO, DATA

SAS

```
data _null_;
  put "Hello from SAS!";
  x = 17.67 * 2;
  put x=;
run;
```

PYTHON

```
print("Hello from Python!")
x = 17.67 * 2
print(x)
```

R

```
print("Hello from R!")
x <- 17.67 * 2
x
```

OUTPUT

```
Hello from ...!
35.34          (SAS writes x=35.34 in the log; R prints [1] 35.34)
```

In SAS, PUT writes to the log, not the results window. Reading the log is a core SAS skill: look for NOTE, WARNING, and ERROR.

WATCH AND EXPLORE

- SAS Programming: Getting Started · free e-learning at learn.sas.com (start with the first module, about 30 minutes)
- R for Data Science (2e), "Introduction" and "Workflow: basics" · free online book at r4ds.hadley.nz (15 minutes)
- DataCamp (optional): Introduction to R, Chapter 1 · datacamp.com (the first chapter of every course is free)

ON THE JOB

Write down one recurring task at work (a monthly score report, a survey summary, an item review). As you go through this year, you'll rebuild pieces of it in code.

KEY IDEA

Question → import → clean → explore → model → communicate. You'll practice every step in SAS, Python, and R.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Name the six steps of the workflow.
2. Where does a SAS PUT statement write?
3. Which language has the psychometrics packages psych, mirt, and lavaan?

More practice: Workbook, Lesson 1.1.

LESSON SUMMARY

Every analysis follows the same workflow; you'll practice it in SAS, Python, and R.

UNIT 1 · LESSON 1.2 · Import · 30 min

Lesson 1.2: Reading Data into SAS, Python, and R

I can import a CSV file in all three languages and check its size and first rows.

WORDS TO KNOW

CSV	Comma-separated values: a plain-text table where commas separate columns.
data frame	A table of rows and columns in Python (pandas) or R; SAS calls it a data set.
working directory	The folder your program looks in when you give a file name without a full path.

WHY IT MATTERS AT WORK

Most analyses fail at step one: the file won't load, columns come in as the wrong type, or the path is wrong. How do you usually check that data loaded correctly?

The data pack's exam.csv holds responses from 600 examinees to a 30-item multiple-choice test (Form A). Each row is one examinee; columns `i01` to `i30` hold the letter each person chose. A blank means the person skipped the item.

CODE IT · IMPORT EXAM.CSV AND LOOK AT IT

SAS

```
%let path = ~/data;      /* the folder where you uploaded the data pack */
proc import datafile="&path/exam.csv" out=work.exam dbms=csv replace;
  getnames=yes;  guessingrows=max;
run;
proc contents data=work.exam varnum; run;
proc print data=work.exam(obs=5); var id form i01-i05; run;
```

PYTHON

```
import pandas as pd
exam = pd.read_csv("data/exam.csv")
print(exam.shape)                          # (rows, columns)
print(exam.loc[:, "id":"i05"].head())
```

R

```
library(readr)                             # part of the tidyverse
exam <- read_csv("data/exam.csv")
dim(exam)                                  # rows, columns
head(exam[, 1:7], 5)
```

OUTPUT

```
600 rows, 32 columns
     id form i01 i02 i03 i04 i05
  E1001    A   B   A   A   D   A
  E1002    A   B   A   A   A   A
  E1003    A   D   A   A   D   A
  E1004    A   B   A   D   D   A
  E1005    A   A   A   A   D   A
```

If SAS can't find the file, right-click the data folder in SAS Studio, choose Properties, and copy the full path into %let path. In Python and R, a relative path like data/exam.csv is read from your working directory.

Always check three things after an import: the number of rows and columns, the first few rows, and the type of each column. Those three checks catch most problems before they spread into your results.

WATCH OUT

Blank cells become missing values: SAS stores a blank character value, while pandas and R store NaN or NA. That will matter when you score the test in Unit 2.

WATCH AND EXPLORE

- R for Data Science (2e), "Data import" · r4ds.hadley.nz (20 minutes)
- Python for Data Analysis (3E), "Data Loading, Storage, and File Formats", reading text files · wesmckinney.com/book (free online) (read the CSV section)
- SAS Programming 1: Essentials, "Accessing Data" lesson · free e-learning at learn.sas.com

KEY IDEA

Import, then immediately check dimensions, the first rows, and column types.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many examinees and columns are in exam.csv?
2. What did examinee E1003 answer on item i01?
3. Which SAS option makes PROC IMPORT scan all rows before guessing column types?

More practice: Workbook, Lesson 1.2.

LESSON SUMMARY

Import, then check dimensions, first rows, and types.

UNIT 1 · LESSON 1.3 · Foundations · 30 min

Lesson 1.3: Variables, Data Types, and Levels of Measurement

I can identify data types and levels of measurement and explain why they decide which analyses make sense.

WORDS TO KNOW

nominal	Categories with no order (program, region).
ordinal	Ordered categories with uneven or unknown spacing (Likert agreement, ranks).
interval	Ordered numbers with equal spacing but no true zero (scaled test scores, °F).
ratio	Equal spacing and a true zero (hours, counts, dollars).
tidy data	Each variable is a column, each observation is a row, each value is a cell.

WHY IT MATTERS AT WORK

A colleague averages survey responses coded 1 = Strongly disagree to 5 = Strongly agree and reports a mean of 3.02. Is that a meaningful number?

Software stores data as types: numbers, text (character or string), logical (true/false), and dates. Statistics also cares about levels of measurement, which describe what the numbers mean.

Level	Examples in the data pack	Sensible summaries
nominal	program, region, gender	counts, percents, mode
ordinal	survey items e1–s4 (1–5 agreement)	medians, percentiles, counts; means used with care
interval	scaled scores (Unit 8), z-scores	means, standard deviations, differences
ratio	hours_prep, study_hours	everything above, plus ratios (“twice as many hours”)

Psychometrics lives on this boundary. Single Likert items are ordinal, but sums of several items often behave approximately like interval scales, which is why analysts routinely average scale scores. Being able to explain that choice is part of being confident in your role.

CODE IT · CHECK COLUMN TYPES

SAS
```
proc import datafile="&path/examinees.csv" out=work.examinees dbms=csv replace;
  getnames=yes;  guessingrows=max;
run;
proc contents data=work.examinees varnum; run;   /* Type: Num or Char */
```

PYTHON
```
examinees = pd.read_csv("data/examinees.csv")
print(examinees.dtypes)
examinees["program"] = examinees["program"].astype("category")
```

R
```
examinees <- read_csv("data/examinees.csv")
str(examinees)                          # or dplyr::glimpse(examinees)
examinees$program <- factor(examinees$program)
```

OUTPUT
```
id, gender, region, program -> text (Char / object or str / chr)
hours_prep -> integer; prior_gpa -> decimal (Num / int64, float64 / dbl)
```

Tidy data makes every analysis easier: one row per observation, one column per variable. Exam data are “wide” (one column per item); for some item analyses you'll reshape them “long” (one row per person-item) in Unit 2.

WATCH AND EXPLORE

- StatQuest: “What is a statistical distribution?” · StatQuest with Josh Starmer on YouTube (about 5 minutes)
- R for Data Science (2e), “Data tidying,” first section · r4ds.hadley.nz (10 minutes)

ON THE JOB

When someone hands you a number, ask what level of measurement it came from. It's one of the fastest ways to spot a summary that doesn't fit the data.

KEY IDEA

Data types tell software how to store values; levels of measurement tell you which statistics are meaningful.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Level of measurement: region
2. Level: a 1–5 agreement item
3. Level: hours_prep

More practice: Workbook, Lesson 1.3.

LESSON SUMMARY

Know your types and levels of measurement before you summarize.

UNIT 1 · LESSON 1.4 · Foundations · 30 min

Lesson 1.4: Reproducible Work Habits

I can organize a project, comment code, read a log, and save outputs so an analysis can be rerun.

WORDS TO KNOW

raw data	The original data exactly as received; never edited by hand.
script	A saved file of code that runs the analysis from start to finish.
version control	A system (such as Git) that records every change to your files.

WHY IT MATTERS AT WORK

Have you ever reopened an old spreadsheet and had no idea how a number was calculated? What would have saved you?

Reproducible work is a habit, not a tool. A few rules make a big difference:

- Keep a project folder: `data/raw`, `data/clean`, `code`, `output`, `docs`.
- Never edit raw data by hand; fix problems in code so the fix is recorded.
- Write a comment for why, not just what.
- Rerun the whole script from the top before you share results.
- Name files so they sort: `2026-11-05_item-analysis.sas`.
- Save the log (SAS) or session information so you know which versions produced the results.

CODE IT · COMMENT, FILTER, AND SAVE A RESULT

SAS

```sas
/* Why: the program office asked for Online examinees only */
data work.online;
  set work.examinees;
  where program = "Online";
run;
proc export data=work.online outfile="&path/online_examinees.csv"
            dbms=csv replace;
run;
```

PYTHON

```python
# Why: the program office asked for Online examinees only
online = examinees[examinees["program"] == "Online"]
online.to_csv("output/online_examinees.csv", index=False)
print(len(online))
```

R

```r
# Why: the program office asked for Online examinees only
online <- subset(examinees, program == "Online")
write_csv(online, "output/online_examinees.csv")
nrow(online)
```

OUTPUT

```
154 rows written
```

Reading the SAS log: NOTE lines report what happened ("The data set WORK.ONLINE has 154 observations"). WARNING lines mean SAS made an assumption you should check. ERROR lines mean a step did not run. Scan the log after every run, the way you'd check rows and columns after an import.

WATCH AND EXPLORE

- R for Data Science (2e), "Workflow: scripts and projects" · r4ds.hadley.nz (15 minutes)
- Happy Git and GitHub for the useR (optional) · happygitwithr.com (skim the introduction)

ON THE JOB

Pick one analysis you'll repeat this year. Create its project folder today, and put the raw file in data/raw with a note about where it came from.

KEY IDEA

Raw data stay untouched; scripts do every change; comments explain why; rerun from the top before sharing.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Should you fix a typo directly in a raw data file?
2. What does a SAS ERROR in the log mean?
3. How many Online examinees are there?

More practice: Workbook, Lesson 1.4.

LESSON SUMMARY

Organize projects, script every change, read the log, and rerun before sharing.

UNIT 1 REVIEW

Unit 1 Study Guide

Use this page to get ready for the unit test. Read each big idea, then try the review pages in your workbook.

Lesson	Remember…
1.1	Every analysis follows the same workflow; you'll practice it in SAS, Python, and R.
1.2	Import, then check dimensions, first rows, and types.
1.3	Know your types and levels of measurement before you summarize.
1.4	Organize projects, script every change, read the log, and rerun before sharing.

UNIT 2

Data Wrangling

Filtering, scoring, grouped summaries, missing data, joins, reshaping, cleaning, validation, SQL, functions, dates, text, and codebooks.

Experienced analysts often say most of the job is getting data ready. In this unit you'll score a 30-item test from raw letters, attach demographics, reshape item data, clean a messy file, query with SQL, and build a pipeline that produces an analysis-ready file with one run. Every step is shown in SAS, Python, and R.

In this unit you will...

- Filter, select, create, sort, and summarize data by group.
- Score multiple-choice responses and compute totals and subscores.
- Handle missing data, join and reshape tables, and clean messy values.
- Write SQL queries, functions, and macros; validate and document data.

ON THE JOB

Scoring, merging, and cleaning are where errors hide. A documented, validated pipeline is one of the most valuable things you can bring to an assessment team.

UNIT VOCABULARY

filter	Keep rows that meet a condition.
left join	Keep all rows of the main table.
long format	One row per person-item.
codebook	Documentation of every variable.
subscore	A score on part of a test.
validation check	A rule the data must satisfy.

Lessons in this unit

Lesson	Title
2.1	Selecting Columns and Filtering Rows

Lesson	Title
2.2	Creating Variables: Scoring a Test
2.3	Sorting, Counting, and Grouped Summaries
2.4	Missing Data: Finding and Handling It
2.5	Joining Tables
2.6	Reshaping Data: Wide and Long
2.7	Cleaning Messy Data
2.8	Duplicates and Validation Checks
2.9	SQL for Analysts I: SELECT, WHERE, GROUP BY
2.10	SQL for Analysts II: Joins and Subqueries
2.11	Don't Repeat Yourself: Functions and Macros
2.12	Putting It Together: A Scoring Pipeline
2.13	Working with Dates
2.14	Working with Text
2.15	Codebooks and Data Documentation
2.16	Wrangling Project: Enrollment Meets Scores

UNIT 2 · LESSON 2.1 · Wrangling · 30 min

Lesson 2.1: Selecting Columns and Filtering Rows

I can keep the columns I need and the rows that meet one or more conditions.

WORDS TO KNOW

filter	Keep only the rows that meet a condition.
select	Keep only certain columns.
Boolean condition	A true/false test, such as hours_prep >= 30.

WHY IT MATTERS AT WORK

A program director asks, “How many Online examinees prepared at least 30 hours?” How would you answer in under a minute?

Almost every question starts by narrowing the data. Filtering keeps rows; selecting keeps columns. Combine conditions with and (both must be true) or or (either can be true).

CODE IT · ONLINE EXAMINEES WITH 30+ HOURS OF PREP

SAS

```
data work.online_hi;
  set work.examinees(keep=id program hours_prep);
  where program = "Online" and hours_prep >= 30;
run;   /* log: WORK.ONLINE_HI has 27 observations */
```

PYTHON

```
online_hi = examinees.loc[
    (examinees["program"] == "Online") & (examinees["hours_prep"] >= 30),
    ["id", "program", "hours_prep"]]
print(len(online_hi))
```

R

```
library(dplyr)
online_hi <- examinees |>
  filter(program == "Online", hours_prep >= 30) |>
  select(id, program, hours_prep)
nrow(online_hi)
```

OUTPUT

```
27
```

In pandas, wrap each condition in parentheses and use & (and) or | (or). In dplyr, commas inside filter() mean “and.”

Task	SAS	Python (pandas)	R (dplyr)
keep columns	KEEP= or KEEP statement	df[[cols]] or .loc	select()
keep rows	WHERE	boolean mask or .query()	filter()
and / or / not	and, or, not	&, |, ~	, or &, |, !
in a list	in ("Day", "Online")	.isin([...])	%in% c(...)

WATCH AND EXPLORE

- R for Data Science (2e), “Data transformation,” rows and columns · r4ds.hadley.nz (20 minutes)
- SAS Programming 1: Essentials, “Preparing Data” · learn.sas.com (free)
- DataCamp (optional): Data Manipulation with pandas, Ch. 1 · datacamp.com

KEY IDEA

Filter rows with conditions, select the columns you need, and check the row count.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many examinees are Online with 30+ hours?
2. pandas operator for “or”
3. dplyr function to keep rows

More practice: Workbook, Lesson 2.1.

LESSON SUMMARY

Filter rows, select columns, and check the count.

UNIT 2 · LESSON 2.2 · Wrangling · Psychometrics · 30 min

Lesson 2.2: Creating Variables: Scoring a Test

I can create new variables, including scoring multiple-choice responses against a key and computing a total score.

WORDS TO KNOW

scoring	Turning responses into points, such as 1 for correct and 0 for incorrect.
array (SAS)	A temporary name for a group of variables so you can loop over them.
vectorized	Applied to a whole column at once instead of one value at a time.

WHY IT MATTERS AT WORK

Your exam data arrive as letters. Before any psychometric analysis, someone has to turn A, B, C, D into 1s and 0s. Who on your team does that, and how?

Scoring is the first transformation in almost every testing project. Each response is compared with the key: a match scores 1, anything else (wrong letter or omitted) scores 0. The total score is the sum of the item scores.

CODE IT · SCORE 30 ITEMS AGAINST THE KEY

SAS

```
data work.scored;
  set work.exam;
  array resp{30} $ i01-i30;
  array key{30} $ _temporary_ ("B" "A" "A" "D" "A" "B" "B" "A" "C" "C" "D" "A" "D" "A" "A" "C"
  array sc{30} s01-s30;
  do j = 1 to 30;
    sc{j} = (resp{j} = key{j});      /* 1 if match, else 0 */
  end;
  total = sum(of s01-s30);
  drop j;
run;
```

PYTHON

```
key = pd.read_csv("data/exam_key.csv")
items = key["item"].tolist()
key_map = dict(zip(key["item"], key["key"]))
scored = pd.DataFrame({it: (exam[it] == k).astype(int)
                       for it, k in key_map.items()})
scored["total"] = scored[items].sum(axis=1)
scored.insert(0, "id", exam["id"])
```

R

```r
key <- read_csv("data/exam_key.csv")
items <- key$item
scored <- as.data.frame(mapply(
  function(x, k) as.integer(!is.na(x) & x == k),
  exam[items], key$key))
scored$total <- rowSums(scored[items])
scored$id <- exam$id
```

OUTPUT

```
first three totals (E1001, E1002, E1003): 29, 22, 27
mean total = 17.67 out of 30
```

WATCH OUT

In R, comparing a missing value gives NA, not FALSE, so an omitted response would make the whole total NA. The `!is.na(x) &` part scores omits as 0. SAS and pandas already treat a blank as "not a match."

ON THE JOB

Ask how omitted responses are scored in your program. Treating omits as wrong, as missing, or as "not reached" changes item statistics, and it should be a documented decision.

WATCH AND EXPLORE

- Python for Data Analysis (3E), "Data Cleaning and Preparation," data transformation · wesmckinney.com/book
- SAS Programming 2 topics: arrays and DO loops (search "SAS arrays" in SAS documentation) · documentation.sas.com

KEY IDEA

Score by comparing each response with its key (1 = match, 0 = otherwise), then sum for a total.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. What is E1001's total score?
2. How is an omitted response scored here?
3. Why add !is.na(x) in R?

More practice: Workbook, Lesson 2.2.

LESSON SUMMARY

Compare each response with the key, then sum the item scores.

UNIT 2 · LESSON 2.3 · Wrangling · 30 min

Lesson 2.3: Sorting, Counting, and Grouped Summaries

I can sort data, count categories, and summarize a variable for each group.

WORDS TO KNOW

group by	Split the data into groups, summarize each, and combine the results.
frequency table	A table of how many rows fall in each category.

WHY IT MATTERS AT WORK

Leaders rarely want 600 rows; they want one line per program. How do you get from rows to that summary?

Split–apply–combine is the pattern behind most reports: split rows into groups, apply a summary to each group, and combine the results into a small table.

CODE IT · HOURS OF PREPARATION BY PROGRAM

SAS

```
proc freq data=work.examinees;
  tables program program*gender / nocol nopercent;
run;
proc means data=work.examinees n mean median maxdec=2;
  class program;
  var hours_prep;
run;
```

PYTHON

```
print(examinees["program"].value_counts())
print(examinees.groupby("program")["hours_prep"]
      .agg(["count", "mean", "median"]).round(2))
```

R

```
count(examinees, program)
examinees |>
  group_by(program) |>
  summarise(n = n(), mean = mean(hours_prep), median = median(hours_prep))
```

OUTPUT

```
program   n    mean   median
Day       283  18.73  18
Evening   163  20.28  19
Online    154  19.84  18
```

Sort with `proc sort` (SAS), `sort_values()` (pandas), or `arrange()` (dplyr). Sorting descending puts the largest values first: in SAS add `descending` before the variable; in pandas use `ascending=False`; in dplyr wrap the variable in `desc()`.

WATCH AND EXPLORE

- R for Data Science (2e), "Data transformation," groups · r4ds.hadley.nz
- Python for Data Analysis (3E), "Data Aggregation and Group Operations" · wesmckinney.com/book
- SAS Programming 1: Essentials, "Analyzing and Reporting on Data" · learn.sas.com (free)

KEY IDEA

Split–apply–combine turns rows into the summary tables leaders need.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many Evening examinees are there?
2. Which program prepared the most hours on average?
3. dplyr function for sorting

More practice: Workbook, Lesson 2.3.

LESSON SUMMARY

Count categories and summarize by group (split–apply–combine).

UNIT 2 · LESSON 2.4 · Wrangling · 30 min

Lesson 2.4: Missing Data: Finding and Handling It

I can count missing values, describe why they matter, and choose a reasonable first strategy.

WORDS TO KNOW

listwise deletion	Dropping any row with a missing value on the variables used.
pairwise deletion	Using all available data for each statistic, so different statistics may use different rows.
missing completely at random (MCAR)	Missingness unrelated to any data, observed or not.

WHY IT MATTERS AT WORK

A survey summary says “n = 800,” but a table inside uses only 536 people. Where did 264 respondents go?

Missing data are normal; ignoring them isn't. First count them (by variable and by row). Then ask why they're missing. Data that are missing completely at random cost you precision; data missing for a reason (for example, unhappy staff skipping workload questions) can bias results.

CODE IT · COUNT MISSING VALUES IN SURVEY.CSV

SAS

```
proc import datafile="&path/survey.csv" out=work.survey dbms=csv replace;
  getnames=yes; guessingrows=max;
run;
proc means data=work.survey n nmiss;
  var e1-e4 w1-w4 s1-s4;
run;
data _null_;
  set work.survey end=last;
  complete + (cmiss(of e1-e4, of w1-w4, of s1-s4) = 0);
  if last then put complete=;
run;
```

PYTHON

```
survey = pd.read_csv("data/survey.csv")
item_cols = [f"{p}{k}" for p in "ews" for k in range(1, 5)]
print(survey[item_cols].isna().sum())
print(survey[item_cols].dropna().shape[0])   # complete rows
```

R

```
survey <- read_csv("data/survey.csv")
item_cols <- c(paste0("e", 1:4), paste0("w", 1:4), paste0("s", 1:4))
colSums(is.na(survey[item_cols]))
sum(complete.cases(survey[item_cols]))
```

OUTPUT

```
missing per item: e1 23, w2 30, s1 31, ... (about 3% each)
complete rows: 536 of 800
```

Strategy	When it's reasonable	Risk
listwise deletion	little missingness, plausibly random	loses many rows when missingness is spread across items
pairwise / available-case	descriptive statistics item by item	statistics based on different people
scale mean of answered items	multi-item scales, e.g., at least 3 of 4 answered	assumes skipped items resemble answered ones
multiple imputation / FIML	modeling with meaningful missingness	more complex; needs care

Notice what happened above: each item is only about 3% missing, yet one third of respondents are missing something. Listwise deletion would throw away 264 people. That's why scale scores are often computed from the items each person answered.

WATCH AND EXPLORE

- R for Data Science (2e), “Missing values” · r4ds.hadley.nz
- Python for Data Analysis (3E), “Handling Missing Data” · wesmckinney.com/book

ON THE JOB

When a report gives an n, ask whether it's everyone or complete cases only, and how missing data were handled. It's one of the most useful questions you can bring to a technical review.

KEY IDEA

Count missingness by variable and by row, think about why it's missing, and document your strategy.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many survey rows are complete on all 12 items?
2. How many responses are missing on w2?
3. What is listwise deletion?

More practice: Workbook, Lesson 2.4.

LESSON SUMMARY

Count missing data, ask why, choose and document a strategy.

UNIT 2 · LESSON 2.5 · Wrangling · 30 min

Lesson 2.5: Joining Tables

I can join two tables on a key and check that the join kept the rows I expected.

WORDS TO KNOW

key	A column that identifies matching rows across tables, such as id.
left join	Keep every row of the left table and add matching columns from the right.
inner join	Keep only rows with a match in both tables.

WHY IT MATTERS AT WORK

Scores live in one file and demographics in another. How do you get mean score by program?

Joins combine tables using a key. A left join keeps every row of your main table (here, the scored exam) and attaches matching information. Always compare row counts before and after: an unexpected change usually means duplicate or missing keys.

CODE IT · ATTACH EXAMINEE INFORMATION TO SCORES

SAS

```
proc sql;
  create table work.scored_info as
  select s.id, s.total, e.program, e.gender, e.region, e.hours_prep
  from work.scored as s left join work.examinees as e
       on s.id = e.id;
quit;
proc means data=work.scored_info mean maxdec=2;
  class program;  var total;
run;
```

PYTHON

```
scored_info = scored[["id", "total"]].merge(
    examinees, on="id", how="left", validate="one_to_one")
print(len(scored_info))
print(scored_info.groupby("program")["total"].mean().round(2))
```

R

```
scored_info <- scored |>
  select(id, total) |>
  left_join(examinees, by = "id")
nrow(scored_info)
scored_info |> group_by(program) |> summarise(mean_total = mean(total))
```

OUTPUT

```
600 rows
Day 17.64   Evening 17.92   Online 17.47
```

pandas' `validate="one_to_one"` stops with an error if either table has duplicate ids, a built-in safety check. In SAS, the classic alternative is a DATA step MERGE with a `BY id` statement after sorting both tables by id.

WATCH AND EXPLORE

- R for Data Science (2e), "Joins" · r4ds.hadley.nz (25 minutes)
- Python for Data Analysis (3E), "Data Wrangling: Join, Combine, and Reshape" · wesmckinney.com/book
- DataCamp (optional): Joining Data with pandas or Joining Data with dplyr · datacamp.com

KEY IDEA

Join on a key, choose the join type on purpose, and check row counts before and after.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. After the left join, how many rows?
2. Mean total for Evening
3. Which join keeps only matches in both tables?

More practice: Workbook, Lesson 2.5.

LESSON SUMMARY

Join on a key and verify the row count.

UNIT 2 · LESSON 2.6 · Wrangling · Psychometrics · 30 min

Lesson 2.6: Reshaping Data: Wide and Long

I can reshape item data from wide (one column per item) to long (one row per person-item) and back.

WORDS TO KNOW

wide format	One row per person, one column per item.
long format	One row per person-item combination.
pivot	Reshape between wide and long.

WHY IT MATTERS AT WORK

To compute statistics for every item at once, which is easier: 30 columns or one column called item?

Exam data usually arrive wide. Many item-level analyses are easier long: one column says which item, one column holds the score, and a grouped summary gives every item's statistics in one step.

CODE IT · WIDE TO LONG, THEN PROPORTION CORRECT BY ITEM

SAS

```
proc sort data=work.scored; by id; run;
proc transpose data=work.scored out=work.long(rename=(col1=score _name_=item));
  by id;
  var s01-s30;
run;
proc means data=work.long mean maxdec=3 nway;
  class item;  var score;
  output out=work.item_p mean=p;
run;
```

PYTHON

```
long = scored.melt(id_vars="id", value_vars=items,
                   var_name="item", value_name="score")
print(long.shape)                                    # (18000, 3)
item_p = long.groupby("item")["score"].mean().round(3)
print(item_p.sort_values().head(3))
```

R

```
library(tidyr)
long <- scored |>
  pivot_longer(all_of(items), names_to = "item", values_to = "score")
dim(long)
long |> group_by(item) |> summarise(p = mean(score)) |> arrange(p) |> head(3)
```

OUTPUT

```
18000 rows (600 people x 30 items)
hardest: i17 0.307, i21 0.315, i13 0.328
```

Going back to wide uses `proc transpose` without the rename, `pivot()` or `pivot_table()` in pandas, and `pivot_wider()` in R.

WATCH AND EXPLORE

- R for Data Science (2e), "Data tidying," pivoting · r4ds.hadley.nz (20 minutes)
- Python for Data Analysis (3E), "Reshaping and Pivoting" · wesmckinney.com/book

KEY IDEA

Long format turns "do this for every item" into one grouped summary.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many rows does the long table have?
2. Which item is hardest?
3. pandas function for wide → long

More practice: Workbook, Lesson 2.6.

LESSON SUMMARY

Pivot between wide and long to fit the analysis.

UNIT 2 · LESSON 2.7 · Wrangling · 30 min

Lesson 2.7: Cleaning Messy Data

I can standardize text categories, convert numbers stored as text, and parse dates in mixed formats.

WORDS TO KNOW

standardize	Make values that mean the same thing look the same.
coerce	Convert to another type, turning values that can't convert into missing.
whitespace	Invisible spaces or tabs, often at the ends of text values.

WHY IT MATTERS AT WORK

Open messy_enrollment.csv. How many different ways is “Online” written?

Real files are messy: Day, day, and DAY (with a trailing space) are the same program; fees appear as 336, $328.00, N/A, or blank; dates arrive as 2026-09-06, 09/04/2026, or Aug 12, 2026. Fix them in code, one column at a time, and check counts after each fix.

CODE IT · CLEAN PROGRAM, STATUS, FEE, AND DATE

SAS

```
data work.clean;
  set work.messy;          /* imported with all columns as text */
  length program_c status_c $10;
  program_c = propcase(strip(program));
  if lowcase(program_c) = "on-line" then program_c = "Online";
  status_c = propcase(strip(status));
  fee_c = input(compress(fee, "$,"), ?? best12.);   /* N/A, blank -> . */
  if index(enroll_date, "/") then date_c = input(enroll_date, mmddyy10.);
  else if anydigit(substr(enroll_date, 1, 1)) then date_c = input(enroll_date, yymmdd10.);
  else date_c = input(cats(scan(enroll_date, 2, " ,"), scan(enroll_date, 1, " "),
                           scan(enroll_date, 3, " ,")), date9.);
  format date_c yymmdd10.;
run;
proc freq data=work.clean; tables program_c status_c; run;
```

PYTHON

```
messy = pd.read_csv("data/messy_enrollment.csv", dtype=str, keep_default_na=False)
messy["program_c"] = (messy["program"].str.strip().str.lower()
                      .replace({"on-line": "online"}).str.title())
messy["status_c"] = messy["status"].str.strip().str.title()
messy["fee_c"] = pd.to_numeric(messy["fee"].str.replace(r"[$,]", "", regex=True),
                               errors="coerce")
messy["date_c"] = pd.to_datetime(messy["enroll_date"], format="mixed")
print(messy["program_c"].value_counts())
```

R

```r
library(stringr); library(lubridate)
messy <- read_csv("data/messy_enrollment.csv", col_types = cols(.default = "c"))
messy <- messy |> mutate(
  program_c = str_to_title(str_trim(str_replace(str_to_lower(program), "on-line", "online"))),
  status_c  = str_to_title(str_trim(status)),
  fee_c     = suppressWarnings(as.numeric(str_remove_all(fee, "[$,]"))),
  date_c    = as_date(parse_date_time(enroll_date, orders = c("Ymd", "mdY", "bdY"))))
count(messy, program_c)
```

OUTPUT

```
program_c: Online 23, Day 21, Evening 19   (63 rows, before removing duplicates)
status_c:  Active 48, Withdrawn 15
fee_c:     10 missing (N/A or blank)
```

WATCH OUT

Read messy files with every column as text first (dtype=str, cols(.default = "c")). Letting software guess types can silently turn “N/A” or “$328.00” into missing values before you see them.

WATCH AND EXPLORE

- R for Data Science (2e), “Strings” and “Dates and times” · r4ds.hadley.nz
- DataCamp (optional): Cleaning Data in Python or Cleaning Data in R · datacamp.com

KEY IDEA

Standardize text, coerce numbers and dates on purpose, and recount after each fix.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many rows are Online after cleaning (before de-duplicating)?
2. How many fees are missing?
3. Why read every column as text first?

More practice: Workbook, Lesson 2.7.

LESSON SUMMARY

Standardize, coerce, parse, and recount.

UNIT 2 · LESSON 2.8 · Wrangling · 30 min

Lesson 2.8: Duplicates and Validation Checks

I can find and remove duplicate records and write checks that catch impossible values.

WORDS TO KNOW

duplicate	A record that appears more than once.
validation check	A rule the data must satisfy, such as scores between 0 and 30.
assertion	A line of code that stops the program if a condition is false.

WHY IT MATTERS AT WORK

If a score file has a duplicated person, what happens to item statistics and pass rates?

Duplicates inflate counts and distort statistics. Exact duplicates (every column identical) are usually copy errors; duplicate ids with different values need investigation. After de-duplicating, write validation checks: rules the clean data must satisfy.

CODE IT · REMOVE EXACT DUPLICATES AND VALIDATE

SAS

```
proc sort data=work.clean out=work.dedup nodupkey;
  by _all_;
run;                       /* log: 3 duplicate observations deleted; 60 remain */
data _null_;
  set work.dedup;
  if program_c not in ("Day", "Evening", "Online") then
    put "CHECK: unexpected program " id= program_c=;
  if not missing(fee_c) and (fee_c < 0 or fee_c > 1000) then
    put "CHECK: fee out of range " id= fee_c=;
run;
```

PYTHON

```
dedup = messy.drop_duplicates()
print(len(messy) - len(dedup), "duplicates removed;", len(dedup), "rows")
assert dedup["id"].is_unique, "duplicate ids remain"
assert dedup["program_c"].isin(["Day", "Evening", "Online"]).all()
assert dedup["fee_c"].dropna().between(0, 1000).all()
```

R

```
dedup <- distinct(messy)
nrow(messy) - nrow(dedup)          # duplicates removed
stopifnot(!anyDuplicated(dedup$id),
          all(dedup$program_c %in% c("Day", "Evening", "Online")),
          all(between(na.omit(dedup$fee_c), 0, 1000)))
```

OUTPUT

```
3 duplicates removed; 60 rows remain (E1004, E1018, E1042 were duplicated)
all checks pass
```

ON THE JOB

Write three validation rules for your next score file: valid ids, scores in range, and one rule only your team would know (for example, Form B examinees must have anchor items).

WATCH AND EXPLORE

- SAS Programming 1: Essentials, “Exploring and Validating Data” · learn.sas.com (free)

KEY IDEA

De-duplicate, then let code check the rules your data must follow.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many exact duplicates were in messy_enrollment?
2. How many rows remain?
3. What does an assertion do when its condition is false?

More practice: Workbook, Lesson 2.8.

LESSON SUMMARY

Remove duplicates and automate validation rules.

UNIT 2 · LESSON 2.9 · SQL · 30 min

Lesson 2.9: SQL for Analysts I: SELECT, WHERE, GROUP BY

I can write SQL queries that filter and summarize a table, and run them from SAS, Python, and R.

WORDS TO KNOW

SQL	Structured Query Language, the standard language for databases.
query	A request for data written in SQL.
aggregate	A summary such as COUNT, AVG, or SUM.

WHY IT MATTERS AT WORK

Much of your organization's data probably lives in a database. What would it change if you could pull exactly what you need yourself?

SQL reads almost like English. The same query runs in SAS (PROC SQL), in Python (here with the built-in sqlite3 module), and in R (with DBI). Learn SQL once and you can use it everywhere.

CODE IT · COUNT AND AVERAGE BY PROGRAM

SAS

```
proc sql;
  select program, count(*) as n, avg(hours_prep) as mean_hours format=6.2
  from work.examinees
  where prior_gpa >= 3.0
  group by program
  order by mean_hours desc;
quit;
```

PYTHON

```
import sqlite3
con = sqlite3.connect(":memory:")
examinees.to_sql("examinees", con, index=False)
q = """
SELECT program, COUNT(*) AS n, ROUND(AVG(hours_prep), 2) AS mean_hours
FROM examinees
WHERE prior_gpa >= 3.0
GROUP BY program
ORDER BY mean_hours DESC"""
print(pd.read_sql(q, con))
```

R

```
library(DBI)
con <- dbConnect(RSQLite::SQLite(), ":memory:")
dbWriteTable(con, "examinees", examinees)
dbGetQuery(con, "
  SELECT program, COUNT(*) AS n, ROUND(AVG(hours_prep), 2) AS mean_hours
  FROM examinees WHERE prior_gpa >= 3.0
  GROUP BY program ORDER BY mean_hours DESC")
```

OUTPUT

```
program   n    mean_hours
Evening   86   20.06
Online    78   19.78
Day      150   18.78
```

Clause	Job	Order SQL runs it
SELECT	columns and summaries to return	5
FROM	which table	1
WHERE	filter rows before grouping	2
GROUP BY	form groups	3
HAVING	filter groups after summarizing	4
ORDER BY	sort the result	6

WATCH AND EXPLORE

- SAS Programming 1: Essentials, “Using SQL in SAS” · learn.sas.com (free)
- DataCamp (optional): Introduction to SQL · datacamp.com

KEY IDEA

SELECT … FROM … WHERE … GROUP BY … HAVING … ORDER BY: one language for every database and all three of your tools.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Which clause filters rows before grouping?
2. Which clause filters groups after summarizing?
3. SAS procedure that runs SQL

More practice: Workbook, Lesson 2.9.

LESSON SUMMARY

SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY.

UNIT 2 · LESSON 2.10 · SQL · 30 min

Lesson 2.10: SQL for Analysts II: Joins and Subqueries

I can join tables and use a subquery in SQL.

WORDS TO KNOW

subquery	A query inside another query.
alias	A short temporary name for a table or column.

WHY IT MATTERS AT WORK

Which examinees scored above the overall average, and what programs are they in?

SQL joins work like the joins in Lesson 2.5. A subquery lets one query use the result of another, such as the overall mean score.

CODE IT · EXAMINEES ABOVE THE MEAN, WITH THEIR PROGRAM

SAS

```
proc sql;
  create table work.above as
  select s.id, s.total, e.program
  from work.scored as s inner join work.examinees as e on s.id = e.id
  where s.total > (select avg(total) from work.scored);
  select program, count(*) as n from work.above group by program;
quit;
```

PYTHON

```
scored[["id", "total"]].to_sql("scored", con, index=False)
q = """
SELECT e.program, COUNT(*) AS n
FROM scored AS s JOIN examinees AS e ON s.id = e.id
WHERE s.total > (SELECT AVG(total) FROM scored)
GROUP BY e.program"""
print(pd.read_sql(q, con))
```

R

```
dbWriteTable(con, "scored", scored[, c("id", "total")])
dbGetQuery(con, "
  SELECT e.program, COUNT(*) AS n
  FROM scored AS s JOIN examinees AS e ON s.id = e.id
  WHERE s.total > (SELECT AVG(total) FROM scored)
  GROUP BY e.program")
```

OUTPUT

```
program   n
Day       147
Evening    91
Online     77
```

KEY IDEA

Join with ON; nest a query in parentheses when one result depends on another.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. What does (SELECT AVG(total) FROM scored) return?
2. What is an alias?
3. Which join keeps only matching rows?

More practice: Workbook, Lesson 2.10.

LESSON SUMMARY

Join with ON; use subqueries for “compared with the overall” questions.

UNIT 2 · LESSON 2.11 · Programming · 30 min

Lesson 2.11: Don't Repeat Yourself: Functions and Macros

I can write a reusable function (Python, R) or macro (SAS) for a task I'd otherwise copy and paste.

WORDS TO KNOW

function	Named, reusable code that takes inputs (arguments) and returns a result.
macro (SAS)	Reusable SAS code with parameters that are filled in before the code runs.
argument	A value you pass into a function.

WHY IT MATTERS AT WORK

How many times have you copied a block of code and changed one name? What happens when you find a bug in the original?

When you copy code more than twice, write a function. A function puts the logic in one place, so a fix happens once. In SAS, the macro language plays this role.

CODE IT · PROPORTION CORRECT FOR ANY ITEM

SAS

```
%macro pcorrect(item);
  proc means data=work.scored mean maxdec=3;
    var &item;
    title "Proportion correct: &item";
  run;
  title;
%mend pcorrect;
%pcorrect(s17)
%pcorrect(s05)
```

PYTHON

```
def p_correct(df, item):
    """Proportion of examinees who answered an item correctly."""
    return round(df[item].mean(), 3)

print(p_correct(scored, "i17"), p_correct(scored, "i05"))
print({it: p_correct(scored, it) for it in items[:3]})
```

R

```
p_correct <- function(df, item) {
  round(mean(df[[item]]), 3)
}
p_correct(scored, "i17"); p_correct(scored, "i05")
sapply(items[1:3], function(it) p_correct(scored, it))
```

OUTPUT

```
i17: 0.307   i05: 0.960
i01 0.415, i02 0.660, i03 0.767
```

The SAS macro writes code for you: `&item` is replaced by whatever you pass in, then the PROC MEANS step runs. Python and R functions receive values and return results directly.

WATCH AND EXPLORE

- R for Data Science (2e), “Functions” · r4ds.hadley.nz (20 minutes)
- SAS Macro Language: search “SAS macro tutorial getting started” in SAS documentation · documentation.sas.com

KEY IDEA

Copy twice, then write a function or macro.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. p_correct(scored, "i17") returns…
2. In SAS, what replaces &item?
3. Why write functions?

More practice: Workbook, Lesson 2.11.

LESSON SUMMARY

Write functions and macros instead of copying code.

UNIT 2 · LESSON 2.12 · Wrangling · Psychometrics · 30 min

Lesson 2.12: Putting It Together: A Scoring Pipeline

I can build an end-to-end script that imports, scores, computes subscores, joins demographics, and exports a clean file.

WORDS TO KNOW

pipeline	A script that runs every step, in order, from raw data to final output.
subscore	A score on part of a test, such as one content domain.

WHY IT MATTERS AT WORK

Imagine new Form A data arrive next month. Could you produce the score file by running one script?

A pipeline strings together everything from this unit. Today's version computes three domain subscores (10 items each) and writes a file ready for analysis.

CODE IT · DOMAIN SUBSCORES AND EXPORT

SAS

```
data work.final;
  merge work.scored(keep=id s01-s30 total) work.examinees;
  by id;
  reasoning   = sum(of s01-s10);
  content     = sum(of s11-s20);
  application = sum(of s21-s30);
run;
proc means data=work.final mean std maxdec=2;
  var reasoning content application total;
run;
proc export data=work.final outfile="&path/scored_formA.csv" dbms=csv replace; run;
```

PYTHON

```
domains = key.groupby("domain")["item"].apply(list)
for d, cols in domains.items():
    scored[d.lower()] = scored[cols].sum(axis=1)
final = scored.merge(examinees, on="id", how="left", validate="one_to_one")
print(final[["reasoning", "content", "application", "total"]].agg(["mean", "std"]).round(2))
final.to_csv("output/scored_formA.csv", index=False)
```

R

```
for (d in unique(key$domain)) {
  scored[[tolower(d)]] <- rowSums(scored[key$item[key$domain == d]])
}
final <- left_join(scored, examinees, by = "id")
final |> summarise(across(c(reasoning, content, application, total), list(mean = mean, sd = sd
write_csv(final, "output/scored_formA.csv")
```

OUTPUT

```
                mean   sd
reasoning       6.38  2.32
content         5.90  2.11
application     5.39  2.33
total          17.67  5.88
```

ON THE JOB

Compare this pipeline with how scoring works on your team. Which steps are automated, and which depend on someone remembering? Those are good places to suggest a script.

KEY IDEA

One script, top to bottom: import, score, subscore, join, validate, export.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Mean Reasoning subscore
2. Which domain has the lowest mean?
3. What does validate="one_to_one" protect against?

More practice: Workbook, Lesson 2.12.

LESSON SUMMARY

Build one script from raw responses to an analysis-ready file.

UNIT 2 · LESSON 2.13 · Wrangling · 30 min

Lesson 2.13: Working with Dates

I can compute differences between dates and summarize by month.

WORDS TO KNOW

date value	A date stored as a number (days since a reference date) so it can be compared and subtracted.
format (SAS)	How a value is displayed, separate from how it is stored.

WHY IT MATTERS AT WORK

How many days before the October 1 test date did each person enroll?

Software stores dates as numbers: SAS counts days from January 1, 1960; R counts from January 1, 1970; pandas stores timestamps. Because they're numbers, you can subtract them to get days between events.

CODE IT · DAYS FROM ENROLLMENT TO THE TEST DATE

SAS

```
data work.dates;
  set work.dedup;
  days_before = '01OCT2026'd - date_c;
  month = month(date_c);
run;
proc means data=work.dates mean min max maxdec=1; var days_before; run;
proc freq data=work.dates; tables month; run;
```

PYTHON

```
dedup["days_before"] = (pd.Timestamp("2026-10-01") - dedup["date_c"]).dt.days
print(dedup["days_before"].describe().round(1))
print(dedup["date_c"].dt.month.value_counts())
```

R

```
dedup <- dedup |> mutate(days_before = as.numeric(as_date("2026-10-01") - date_c),
                         month = month(date_c))
summary(dedup$days_before)
count(dedup, month)
```

OUTPUT

```
E1001 enrolled 2026-09-04: 27 days before the test
month 8 (August): 40 enrollments   month 9 (September): 20
```

KEY IDEA

Dates are numbers underneath, so you can subtract, compare, and group them.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many days before Oct 1 did E1001 enroll?

2. Earliest enrollment date
3. SAS date constant for Oct 1, 2026

More practice: Workbook, Lesson 2.13.

LESSON SUMMARY

Subtract dates for durations; extract parts to group.

UNIT 2 · LESSON 2.14 · Wrangling · 30 min

Lesson 2.14: Working with Text

I can extract, change, and search text values, including simple patterns.

WORDS TO KNOW

string	A text value.
substring	Part of a string.
regular expression	A pattern for matching text, such as \d+ for one or more digits.

WHY IT MATTERS AT WORK

Ids like E1001 hide a number. Comment fields hide themes. How do you get at what's inside text?

CODE IT · EXTRACT THE NUMBER FROM AN ID AND SEARCH TEXT

SAS

```
data work.ids;
  set work.examinees(keep=id);
  id_num = input(substr(id, 2), 8.);
  is_formA = (char(id, 1) = "E" and id_num < 3000);
run;
data work.withdrew;
  set work.messy;
  if prxmatch("/withdr/i", status);    /* case-insensitive pattern */
run;
```

PYTHON

```
examinees["id_num"] = examinees["id"].str[1:].astype(int)
withdrew = messy[messy["status"].str.contains("withdr", case=False)]
print(examinees["id_num"].min(), examinees["id_num"].max(), len(withdrew))
```

R

```
examinees$id_num <- as.integer(str_sub(examinees$id, 2))
withdrew <- messy |> filter(str_detect(status, regex("withdr", ignore_case = TRUE)))
range(examinees$id_num); nrow(withdrew)
```

OUTPUT

```
id numbers 1001 to 1600; 15 withdrawn rows (before removing duplicates)
```

Regular expressions look cryptic but a few cover most needs: `\d` a digit, `+` one or more, `^` start, `$` end, `[A-Z]` any capital letter.

WATCH AND EXPLORE

- R for Data Science (2e), “Regular expressions” · r4ds.hadley.nz

KEY IDEA

Use substrings and patterns to pull meaning out of text, then convert to numbers when needed.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Number part of E1234
2. What does \d+ match?
3. Withdrawn rows (raw, with duplicates)

More practice: Workbook, Lesson 2.14.

LESSON SUMMARY

Substrings and regular expressions unlock text.

UNIT 2 · LESSON 2.15 · Documentation · 30 min

Lesson 2.15: Codebooks and Data Documentation

I can write a codebook that explains every variable in a data set.

WORDS TO KNOW

codebook	A document listing each variable's name, meaning, type, allowed values, and how it was made.
label	A longer, readable description attached to a variable.
value label / format	Text shown in place of a code, such as 1 = Strongly disagree.

WHY IT MATTERS AT WORK

If you left your job tomorrow, could someone else understand your score file?

A codebook turns a data file into something others can trust and reuse. For each variable record: name, label, type, allowed values or range, missing-data codes, and how it was created (source or formula).

Variable	Label	Type	Values	Created from
total	Total raw score, Form A	integer	0–30	sum of s01–s30; omits scored 0
reasoning	Reasoning subscore	integer	0–10	sum of s01–s10
w3	I have enough time to finish my work	ordinal	1–5; blank = missing	reverse-worded; reverse before scoring

CODE IT · ATTACH LABELS AND VALUE FORMATS

SAS

```
proc format;
  value agree 1="Strongly disagree" 2="Disagree" 3="Neutral" 4="Agree" 5="Strongly agree";
run;
data work.survey_lab;
  set work.survey;
  label e1 = "I feel energized by my work"  w3 = "I have enough time to finish my work";
  format e1-e4 w1-w4 s1-s4 agree.;
run;
proc contents data=work.survey_lab varnum; run;   /* a starter codebook */
```

PYTHON

```
codebook = pd.DataFrame({
    "variable": ["total", "reasoning", "w3"],
    "label": ["Total raw score, Form A", "Reasoning subscore", "I have enough time to finish m
    "type": ["integer", "integer", "ordinal"],
    "values": ["0-30", "0-10", "1-5; blank = missing"]})
codebook.to_csv("docs/codebook.csv", index=False)
```

R

```
codebook <- tibble::tribble(
  ~variable,   ~label,                                ~type,     ~values,
  "total",     "Total raw score, Form A",             "integer", "0-30",
  "reasoning", "Reasoning subscore",                  "integer", "0-10",
  "w3",        "I have enough time to finish my work", "ordinal", "1-5; blank = missing")
write_csv(codebook, "docs/codebook.csv")
```

OUTPUT

```
a codebook table saved to docs/codebook.csv (SAS: labels and formats appear in PROC CONTENTS)
```

ON THE JOB

Start a codebook for one file you use at work. Even five well-documented variables will make your next analysis faster.

KEY IDEA

Document every variable: name, label, type, values, missing codes, and origin.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. Name four things a codebook records.
2. What is a SAS format?
3. Why note that w3 is reverse-worded?

More practice: Workbook, Lesson 2.15.

LESSON SUMMARY

A codebook makes data understandable and reusable.

UNIT 2 · LESSON 2.16 · Project · 30 min

Lesson 2.16: Wrangling Project: Enrollment Meets Scores

I can clean, de-duplicate, join, validate, document, and summarize a messy file from start to finish.

WHY IT MATTERS AT WORK

This is the kind of request that lands on your desk: “Can you tell us how withdrawn students were doing?”

Combine everything from Unit 2. Work in the language you're least comfortable with; then check your numbers in a second language.

- Clean messy_enrollment.csv (program, status, fee, date) and remove exact duplicates (60 rows).
- Join the cleaned file to scored_formA.csv by id (left join; keep all 60).
- Validate: unique ids, valid programs, fees in range.
- Summarize mean total and mean hours_prep for Active vs Withdrawn.
- Write a three-row codebook for your new variables and a two-sentence summary for a non-technical reader.

KEY IDEA

Real projects use every wrangling verb. A clean, documented pipeline is the foundation for everything that follows.

QUICK CHECK

Answer from memory first, then check the answer key at the back of your workbook.

1. How many rows after de-duplicating?
2. Which join keeps all enrollment rows?
3. Name one validation check.

More practice: Workbook, Lesson 2.16.

LESSON SUMMARY

One messy file, cleaned and joined, start to finish.

UNIT 2 REVIEW

Unit 2 Study Guide

Use this page to get ready for the unit test. Read each big idea, then try the review pages in your workbook.

Lesson	Remember…
2.1	Filter rows, select columns, and check the count.
2.2	Compare each response with the key, then sum the item scores.
2.3	Count categories and summarize by group (split–apply–combine).
2.4	Count missing data, ask why, choose and document a strategy.
2.5	Join on a key and verify the row count.
2.6	Pivot between wide and long to fit the analysis.
2.7	Standardize, coerce, parse, and recount.
2.8	Remove duplicates and automate validation rules.
2.9	SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY.
2.10	Join with ON; use subqueries for “compared with the overall” questions.
2.11	Write functions and macros instead of copying code.
2.12	Build one script from raw responses to an analysis-ready file.
2.13	Subtract dates for durations; extract parts to group.
2.14	Substrings and regular expressions unlock text.
2.15	A codebook makes data understandable and reusable.
2.16	One messy file, cleaned and joined, start to finish.

Glossary

aggregate	A summary such as COUNT, AVG, or SUM.
alias	A short temporary name for a table or column.
argument	A value you pass into a function.
array (SAS)	A temporary name for a group of variables so you can loop over them.
assertion	A line of code that stops the program if a condition is false.
Boolean condition	A true/false test, such as hours_prep >= 30.
codebook	Documentation of every variable.
coerce	Convert to another type, turning values that can't convert into missing.
CSV	Comma-separated values: a plain-text table where commas separate columns.
data frame	A table of rows and columns.
date value	A date stored as a number (days since a reference date) so it can be compared and subtracted.
duplicate	A record that appears more than once.
filter	Keep rows that meet a condition.
format (SAS)	How a value is displayed, separate from how it is stored.
frequency table	A table of how many rows fall in each category.
function	Named, reusable code that takes inputs (arguments) and returns a result.
group by	Split the data into groups, summarize each, and combine the results.
IDE	Integrated development environment: the program where you write and run code (SAS Studio, RStudio, Jupyter, VS Code).
inner join	Keep only rows with a match in both tables.
interval	Ordered numbers with equal spacing but no true zero (scaled test scores, °F).
key	A column that identifies matching rows across tables, such as id.
label	A longer, readable description attached to a variable.
left join	Keep all rows of the main table.
listwise deletion	Dropping any row with a missing value on the variables used.
long format	One row per person-item.
macro (SAS)	Reusable SAS code with parameters that are filled in before the code runs.

missing completely at random (MCAR)	Missingness unrelated to any data, observed or not.
nominal	Categories with no order (program, region).
ordinal	Ordered categories.
pairwise deletion	Using all available data for each statistic, so different statistics may use different rows.
pipeline	A script that runs every step, in order, from raw data to final output.
pivot	Reshape between wide and long.
query	A request for data written in SQL.
ratio	Equal spacing and a true zero (hours, counts, dollars).
raw data	The original data exactly as received; never edited by hand.
regular expression	A pattern for matching text, such as \d+ for one or more digits.
reproducible	Can be rerun with the same result.
scoring	Turning responses into points, such as 1 for correct and 0 for incorrect.
script	A saved file of code that runs the analysis from start to finish.
select	Keep only certain columns.
SQL	Structured Query Language, the standard language for databases.
standardize	Make values that mean the same thing look the same.
string	A text value.
subquery	A query inside another query.
subscore	A score on part of a test.
substring	Part of a string.
tidy data	Each variable is a column, each observation is a row, each value is a cell.
validation check	A rule the data must satisfy.
value label / format	Text shown in place of a code, such as 1 = Strongly disagree.
vectorized	Applied to a whole column at once instead of one value at a time.
version control	A system (such as Git) that records every change to your files.
whitespace	Invisible spaces or tabs, often at the ends of text values.
wide format	One row per person, one column per item.
workflow	Steps from question to communication.
working directory	The folder your program looks in when you give a file name without a full path.

Data Science & Psychometrics

Student Textbook · 2026–2027 · Professional Learning

A year of professional learning in statistics, data science, and psychometrics, built for a working professional: about 30 minutes a day, Monday through Friday. Every lesson pairs a short reading with worked examples, a suggested video, and hands-on code in SAS, Python, and R, using realistic assessment and survey data.

This Textbook teaches every lesson: the big idea, key terms, worked examples, side-by-side code in SAS, Python, and R, suggested videos, and On the Job notes that connect each skill to real work. It pairs with the Workbook.

INSIDE THIS YEAR

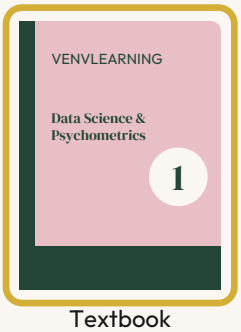
01 Your Data Toolkit

02 Data Wrangling

IN THIS BOOK

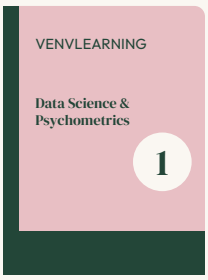
- Four 30-minute lessons a week, Monday–Thursday
- Code It examples in SAS, Python, and R
- Watch lists: videos and free courses for each topic
- On the Job notes and a glossary

THE COMPLETE SET



Textbook

You are holding this book



Workbook

Each lesson has the same number in both books, and every day of the year is mapped to its lesson in the VenvLearning planner.